

Documento Técnico: CÉNIT - Sistema de Reservas de Hotel

Administración de Bases de Datos – Proyecto Final (Grupo V)

Desarrollado por:

Josué Abdel Ortiz Deodanes - 00017125
Osmar Alejandro Rodas Rodriguez - 00068125
Kevin Vladimir Rodriguez Alvarado - 00039425
Anderson José Turcios Fores - 00003925
Josué Emiliano Valdés Jacobo - 00003525

1. Descripción del sistema y reglas de negocio

El sistema CÉNIT está diseñado para administrar integralmente las operaciones de un hotel: gestión de habitaciones y registro de huéspedes, control de reservaciones, seguimiento de consumos de servicios adicionales y emisión de facturación detallada por estancia.

Este proyecto no parte desde cero: retoma la base desarrollada en el ciclo anterior para la materia de Bases de Datos, cuyo esquema operaba sobre PostgreSQL. El principal reto técnico fue la migración a SQL Server, que implicó adaptar tipos de datos incompatibles, reescribir funciones y triggers a T-SQL y resolver diferencias en la gestión de secuencias y restricciones de integridad. En esta etapa de Administración de Bases de Datos se agregan seguridad por esquemas y roles, auditoría, optimización con índices, dimensionamiento, respaldos automatizados y un dashboard analítico.

Reglas de negocio

- **Historial de clientes:** un huésped puede realizar múltiples reservaciones a lo largo del tiempo.
- **Cobertura de reserva:** una reservación cubre una o más habitaciones por un rango de fechas definido (entrada y salida).
- **Consumos adicionales:** durante la estancia (check-in a check-out) el huésped puede consumir servicios del hotel (restaurante, lavandería, spa, etc.).
- **Facturación:** al hacer check-out se genera una factura consolidada con el cobro de la habitación y los servicios, aplicando las reglas de temporadas (aumentos) y descuentos.
- **Restricción crítica:** una habitación no puede estar reservada ni ocupada dos veces en el mismo período (solapamiento de fechas); la valida el trigger `trg_verificar_reserva`.

Resumen técnico de la solución

Elemento	Detalle
Motor / base de datos	Microsoft SQL Server · base de datos `HotelDB`
Modelo físico	18 tablas en 3FN distribuidas en 4 esquemas de datos (`catalogo`, `operacion`, `facturacion`, `calidad`) y un esquema `reportes` para vistas analíticas
Lógica de negocio	1 función (`fn_buscar_habitaciones_disponibles`), 1 procedimiento (`sp_calcular_total_factura`) y 4 triggers
Capa analítica	13 vistas en `reportes`, 3 vistas de análisis con funciones ventana y 3 vistas indexadas en `facturacion`
Seguridad	7 roles de base de datos, permisos por esquema, DENY explícito al auditor y SQL Server Audit sobre facturación
Continuidad	Recovery FULL; respaldos Full semanal, Diferencial diario y Log cada 15 min automatizados con SQL Server Agent

Nota: los scripts de seguridad y auditoría del proyecto hacen referencia a la misma base con el nombre `cenitDB`; en este documento se usa `HotelDB`, nombre empleado en los scripts de carga, optimización y respaldo.

2. Modelo entidad-relación y esquema relacional

A continuación se presenta el diagrama entidad-relación en notación Chen (Figura 1) y el esquema relacional normalizado en Tercera Forma Normal (Figura 2), que elimina dependencias parciales y transitivas. Las llaves primarias se denotan con **(PK)** y las foráneas con **(FK)**. Las figuras se muestran en las páginas siguientes, a página completa para conservar su legibilidad; las relaciones principales se resumen en la sección 2.3.

Versión interactiva de ambos diagramas en draw.io: diagramas_cenit

2.1. Diagrama entidad-relación (notación Chen)

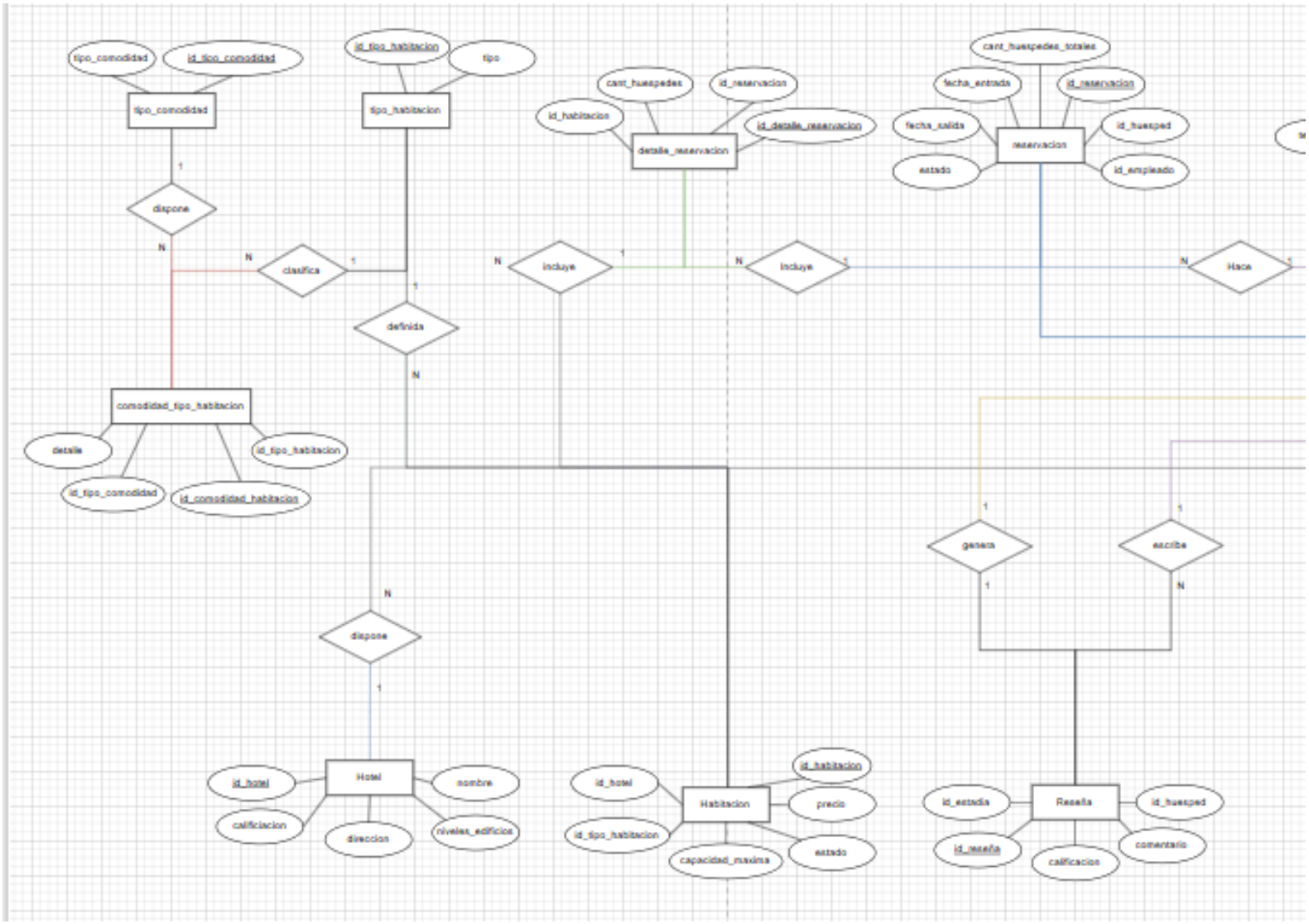


Figura 1a. Diagrama ER en notación Chen (parte 1 de 2).

2.1. Diagrama entidad-relación (continuación)

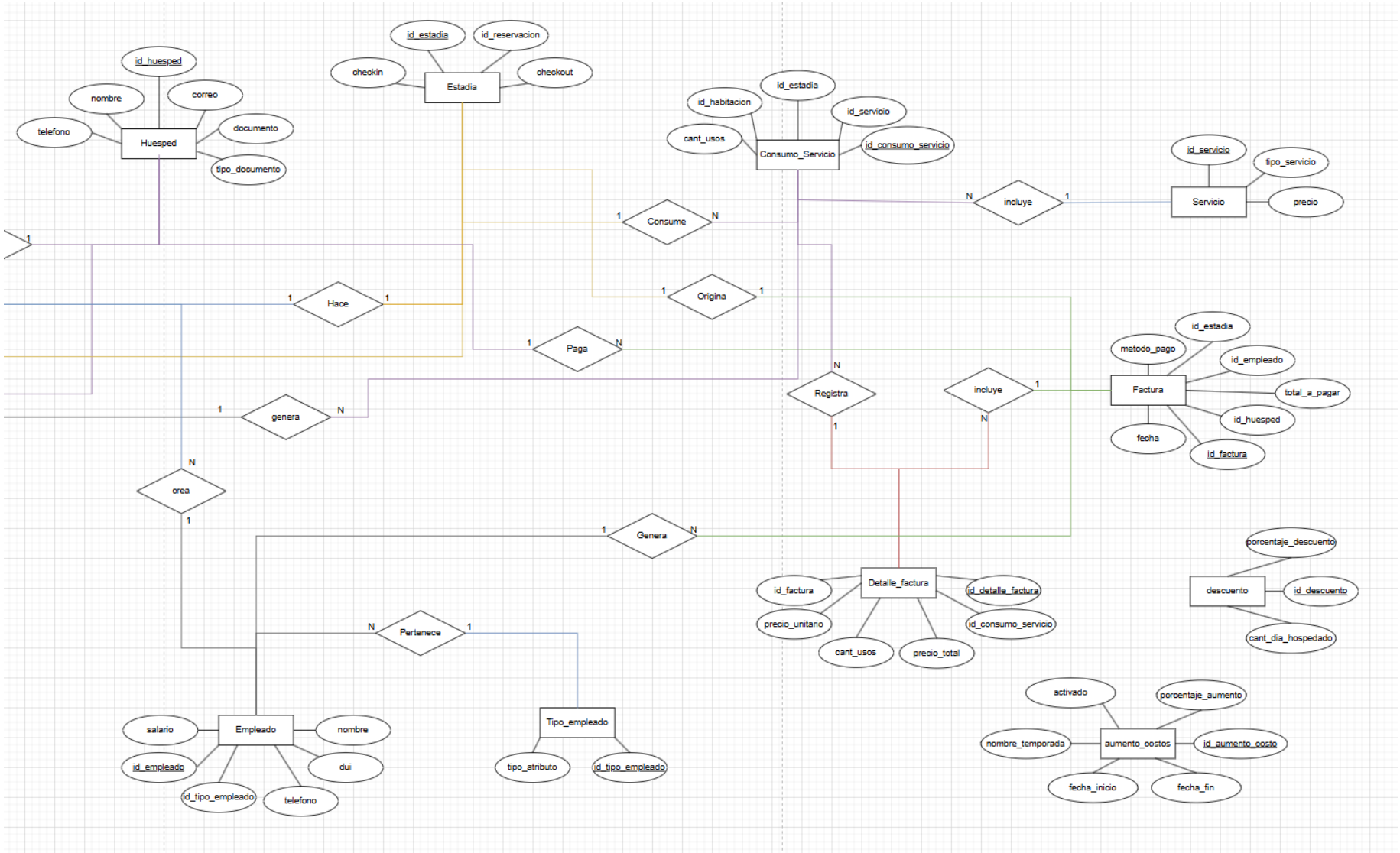


Figura 1b. Diagrama ER en notación Chen (parte 2 de 2).

2.2. Esquema relacional normalizado (3FN)

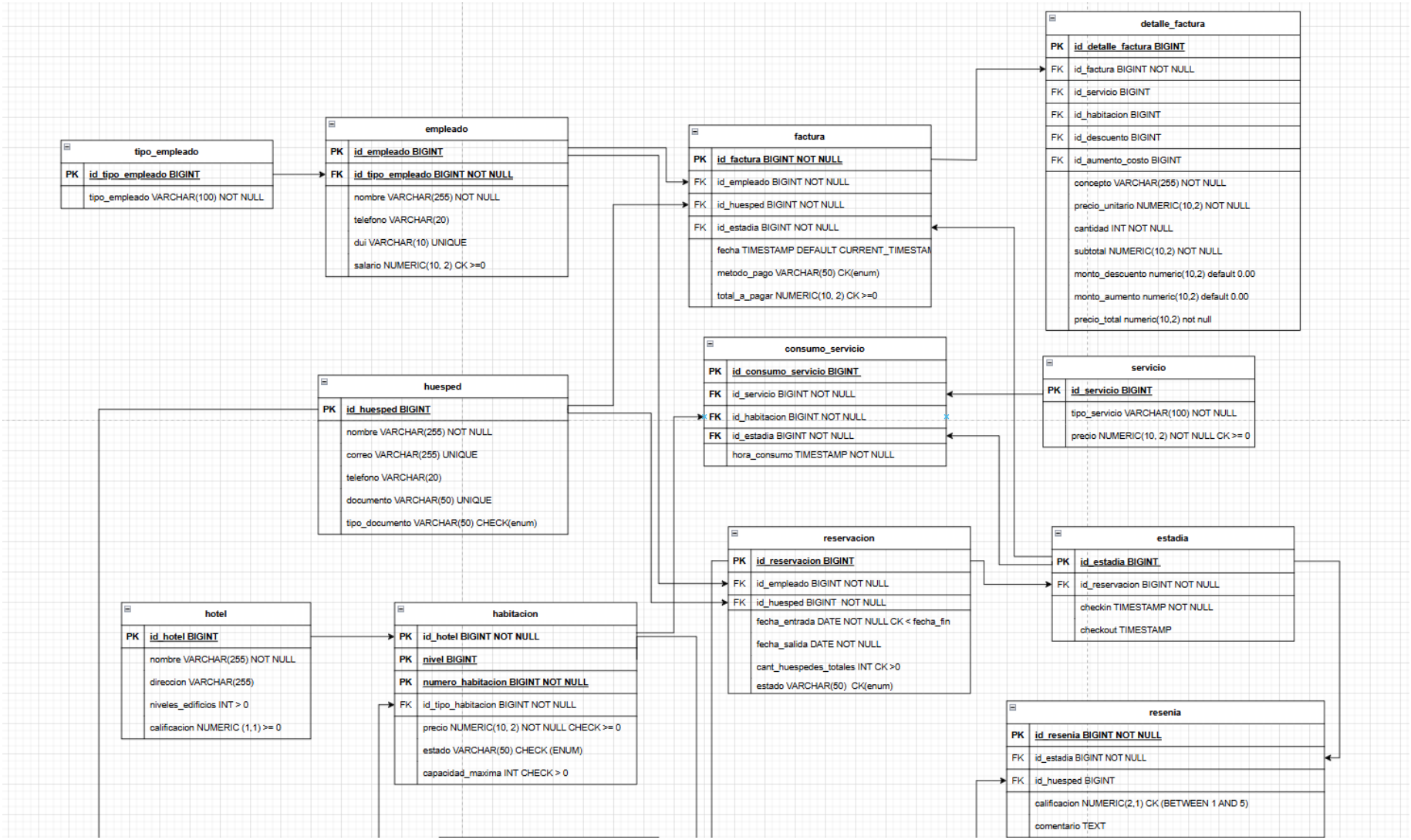


Figura 2a. Esquema relacional en 3FN (parte 1 de 2).

2.2. Esquema relacional normalizado (continuación)

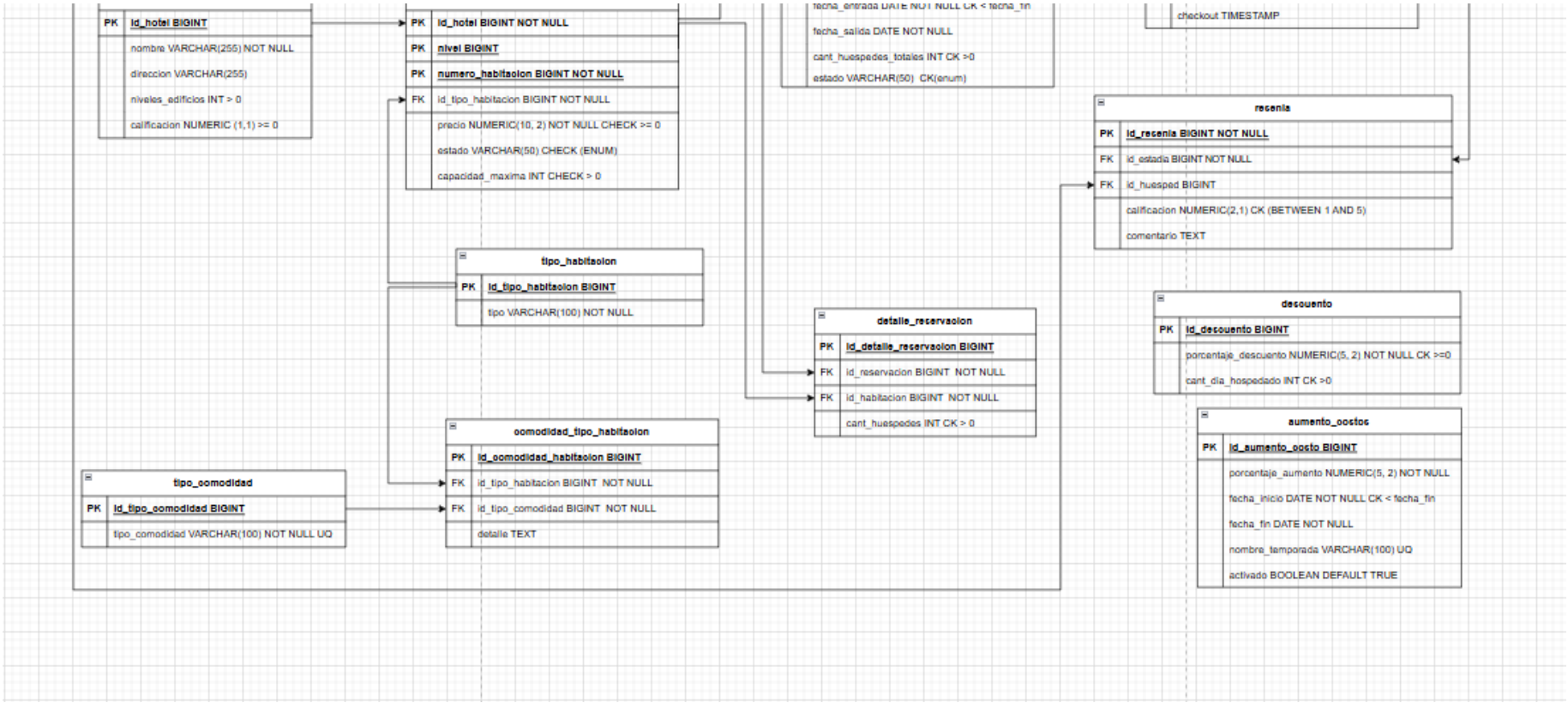


Figura 2b. Esquema relacional en 3FN (parte 2 de 2).

2.3. Relaciones principales del modelo

Relación	Cardinalidad	Implementación
hotel – habitacion	1 : N	`habitacion.id_hotel`; nivel limitado por `niveles_edificios` (trigger)
tipo_habitacion – habitacion	1 : N	`habitacion.id_tipo_habitacion`
tipo_habitacion – tipo_comodidad	N : M	Tabla intermedia `comodidad_tipo_habitacion`
huesped – reservacion	1 : N	`reservacion.id_huesped`; la gestiona un `empleado`
reservacion – habitacion	N : M	Tabla intermedia `detalle_reservacion` (fechas de entrada y salida)
reservacion – estadia	1 : N	`estadia.id_reservacion` (check-in / check-out)
estadia – servicio	N : M	Tabla `consumo_servicio` (servicio, habitación y estadía)
estadia – factura	1 : N	`factura` única por (empleado, huésped, estadía); trigger `trg_checkout_factura`
factura – detalle_factura	1 : N	Cada línea cobra un servicio o una habitación (`ck_origen_cobro`)
estadia / huesped – resenia	1 : N	Una reseña por (estadía, huésped); recalcula `hotel.calificacion`

3. Diccionario de datos

El modelo físico de CÉNIT en SQL Server se compone de 18 tablas organizadas en cuatro esquemas según su función, lo que permite otorgar permisos por esquema en lugar de tabla por tabla. En cada tabla, la columna «Llave / Nulo» indica si el campo es llave primaria (PK) o foránea (FK, con la tabla a la que referencia) y si admite valores nulos.

Esquema	Propósito	Tablas
catalogo	Datos de referencia y configuración, de baja frecuencia de cambio.	hotel, tipo_habitacion, tipo_comodidad, tipo_empleado, servicio, descuento, aumento_costos, comodidad_tipo_habitacion
operacion	Operación diaria: habitaciones, personal, huéspedes y ciclo de reservación y estadía.	habitacion, empleado, huesped, reservacion, detalle_reservacion, estadia, consumo_servicio
facturacion	Emisión y detalle de las facturas.	factura, detalle_factura
calidad	Opiniones de los huéspedes sobre su estadía.	resenia

3.1. Tabla: catalogo.hotel

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_hotel`	bigint	PK / No Nulo	Identity	Identificador único del hotel.
`nombre`	nvarchar(255)	No Nulo	Unique	Nombre comercial del hotel.
`direccion`	nvarchar(255)	Nulo	-	Ubicación física del hotel.
`niveles_edificios`	int	No Nulo	Check (> 0)	Cantidad de niveles del edificio; limita el nivel de sus habitaciones (trigger tg_check_nivel_habitacion).
`calificacion`	decimal(2,1)	Nulo	Check (entre 1 y 5)	Promedio de las reseñas del hotel; se recalcula con el trigger trg_actualizar_calificacion.
`descripcion`	nvarchar(max)	Nulo	-	Descripción general del hotel.

3.2. Tabla: catalogo.tipo_habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_tipo_habitacion`	bigint	PK / No Nulo	Identity	Identificador único del tipo de habitación.
`tipo_habitacion`	nvarchar(100)	No Nulo	Unique	Nombre del tipo de habitación (p. ej., Suite Ejecutiva).

3.3. Tabla: catalogo.tipo_comodidad

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_tipo_comodidad`	bigint	PK / No Nulo	Identity	Identificador único de la comodidad.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
tipo_comodidad	nvarchar(100)	No Nulo	Unique	Nombre de la comodidad (p. ej., Jacuzzi Privado).

3.4. Tabla: catalogo.tipo_empleado

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_tipo_empleado	bigint	PK / No Nulo	Identity	Identificador único del tipo de empleado.
tipo_empleado	nvarchar(100)	No Nulo	Unique	Rol o cargo del empleado (p. ej., Recepcionista, Gerente).

3.5. Tabla: catalogo.servicio

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_servicio	bigint	PK / No Nulo	Identity	Identificador único del servicio.
tipo_servicio	nvarchar(100)	No Nulo	Unique	Nombre del servicio adicional (p. ej., Lavandería).
precio	decimal(10,2)	No Nulo	Check (>= 0)	Precio unitario del servicio.

3.6. Tabla: catalogo.descuento

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_descuento	bigint	PK / No Nulo	Identity	Identificador único del descuento.
porcentaje_descuento	decimal(5,2)	No Nulo	Check (>= 0 AND <= 100)	Porcentaje de descuento aplicable a la estadía.
cant_dia_hospedado	int	Nulo	Check (> 0)	Noches mínimas para que aplique el descuento; al facturar se usa el umbral más alto que no supere la estadía.

3.7. Tabla: catalogo.aumento_costos

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_aumento_costo	bigint	PK / No Nulo	Identity	Identificador único del aumento por temporada.
porcentaje_aumento	decimal(5,2)	No Nulo	Check (>= 0)	Porcentaje a incrementar sobre la tarifa base.
fecha_inicio	date	No Nulo	Check (fecha_inicio < fecha_fin)	Fecha de inicio de la temporada (inclusive).
fecha_fin	date	No Nulo	-	Fecha de finalización de la temporada (inclusive).
nombre_temporada	nvarchar(100)	Nulo	Unique	Nombre descriptivo de la temporada (p. ej., Semana Santa 2026).
activado	bit	No Nulo	Default (1)	Solo las temporadas activas se consideran al calcular la factura.

3.8. Tabla: catalogo.comodidad_tipo_habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_comodidad_habitacion	bigint	PK / No Nulo	Identity	Identificador único de la asociación.
id_tipo_habitacion	bigint	FK / No Nulo → catalogo.tipo_habitacion	-	Tipo de habitación al que pertenece la comodidad.
id_tipo_comodidad	bigint	FK / No Nulo → catalogo.tipo_comodidad	-	Comodidad que ofrece el tipo de habitación.
detalle	nvarchar(max)	No Nulo	-	Condición de la comodidad (p. ej., Incluido en la tarifa).

Nota: Asocia comodidades a un tipo de habitación, no a una habitación individual; por eso no referencia a operacion.habitacion.

3.9. Tabla: operacion.habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_habitacion	bigint	PK / No Nulo	Identity	Identificador único de la habitación.
id_hotel	bigint	FK / No Nulo → catalogo.hotel	-	Hotel al que pertenece la habitación.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
nivel	int	No Nulo	Check (>= 0)	Nivel del edificio donde se ubica; no puede superar los niveles del hotel.
numero_habitacion	int	No Nulo	Check (>= 0)	Número de la habitación dentro del nivel.
id_tipo_habitacion	bigint	FK / No Nulo → catalogo.tipo_habitacion	-	Clasificación de la habitación.
precio	decimal(10,2)	No Nulo	Check (>= 0)	Tarifa por noche.
estado	nvarchar(50)	No Nulo	Check (DISPONIBLE, OCUPADA, MANTENIMIENTO)	Estado operativo de la habitación.
capacidad_maxima	int	No Nulo	Check (entre 1 y 15)	Cantidad máxima de huéspedes.
descripcion	nvarchar(max)	Nulo	-	Descripción de la habitación.

Nota: Restricción Unique compuesta: (id_hotel, nivel, numero_habitacion).

3.10. Tabla: operacion.empleado

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_empleado	bigint	PK / No Nulo	Identity	Identificador único del empleado.
id_tipo_empleado	bigint	FK / No Nulo → catalogo.tipo_empleado	-	Cargo del empleado.
nombre	nvarchar(255)	No Nulo	-	Nombre completo del empleado.
correo	nvarchar(100)	No Nulo	Unique, Check (formato de correo)	Correo electrónico institucional.
telefono	nvarchar(20)	Nulo	Check (+503 seguido de 2, 6 o 7 y 7 dígitos)	Teléfono de contacto salvadoreño.
dui	nvarchar(10)	No Nulo	Unique, Check (8 dígitos, guion, 1 dígito)	Documento Único de Identidad.
salario	decimal(10,2)	No Nulo	Check (>= 0)	Salario mensual del empleado.

3.11. Tabla: operacion.huesped

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_huesped	bigint	PK / No Nulo	Identity	Identificador único del huésped.
nombre	nvarchar(255)	No Nulo	-	Nombre completo del huésped.
correo	nvarchar(255)	No Nulo	Unique, Check (formato de correo)	Correo electrónico de contacto.
telefono	nvarchar(20)	No Nulo	Check (7 a 20 caracteres: dígitos, espacios, guion o +)	Teléfono de contacto.
documento	nvarchar(50)	No Nulo	Unique	Número del documento de identidad; su formato depende del tipo de documento.
tipo_documento	nvarchar(50)	No Nulo	Check (DUI, PASAPORTE, CONSTANCIA DE RESIDENCIA)	Tipo de documento presentado.

3.12. Tabla: operacion.reservacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_reservacion	bigint	PK / No Nulo	Identity	Identificador único de la reservación.
id_empleado	bigint	FK / No Nulo → operacion.empleado	-	Empleado que gestionó la reservación.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped que solicita la reservación.
cant_huespedes_totales	int	No Nulo	Check (> 0)	Total de huéspedes de la reservación.
estado	nvarchar(50)	No Nulo	Check (PENDIENTE, CONFIRMADA, CANCELADA, RECHAZADA, COMPLETADA)	Estado actual de la reservación.

3.13. Tabla: operacion.detalle_reservacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_detalle_reservacion	bigint	PK / No Nulo	Identity	Identificador único del detalle.
id_reservacion	bigint	FK / No Nulo → operacion.reservacion	-	Reservación a la que pertenece.
id_habitacion	bigint	FK / No Nulo → operacion.habitacion	-	Habitación reservada.
cant_huespedes	int	No Nulo	Check (> 0)	Huéspedes alojados en esta habitación.
fecha_entrada	date	No Nulo	-	Fecha de entrada.
fecha_salida	date	No Nulo	Check (fecha_entrada < fecha_salida)	Fecha de salida.

Nota: El trigger trg_verificar_reserva rechaza el registro si la habitación ya tiene otra reserva activa (no cancelada ni rechazada) con fechas que se traslapan.

3.14. Tabla: operacion.estadia

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_estadia	bigint	PK / No Nulo	Identity	Identificador único de la estadía.
id_reservacion	bigint	FK / No Nulo → operacion.reservacion	-	Reservación que origina la estadía.
checkin	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora de ingreso.
checkout	datetime2(3)	Nulo	Check (nulo o checkin < checkout)	Fecha y hora de salida; al registrarla, el trigger trg_checkout_factura genera la factura.

3.15. Tabla: operacion.consumo_servicio

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_consumo_servicio	bigint	PK / No Nulo	Identity	Identificador único del consumo.
id_servicio	bigint	FK / No Nulo → catalogo.servicio	-	Servicio consumido.
id_habitacion	bigint	FK / No Nulo → operacion.habitacion	-	Habitación desde la que se consumió.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía a la que se carga el consumo.
hora_consumo	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora del consumo.

3.16. Tabla: facturacion.factura

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_factura	bigint	PK / No Nulo	Identity	Identificador único de la factura.
id_empleado	bigint	FK / No Nulo → operacion.empleado	-	Empleado que emite la factura.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped facturado.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía que se factura.
fecha	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora de emisión.
metodo_pago	nvarchar(50)	No Nulo	Check (EFECTIVO, TRANSFERENCIA, TARJETA, BITCOIN, PAYPAL)	Forma de pago utilizada.
total_a_pagar	decimal(10,2)	No Nulo	Check (>= 0)	Suma de precio_total de su detalle; la calcula sp_calcular_total_factura.

Nota: Restricción Unique compuesta: (id_empleado, id_huesped, id_estadia).

3.17. Tabla: facturacion.detalle_factura

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_detalle_factura	bigint	PK / No Nulo	Identity	Identificador único de la línea de factura.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_factura	bigint	FK / No Nulo → facturacion.factura	-	Factura a la que pertenece la línea.
id_servicio	bigint	FK / Nulo → catalogo.servicio	-	Servicio cobrado (línea de servicio).
id_habitacion	bigint	FK / Nulo → operacion.habitacion	-	Habitación cobrada (línea de hospedaje).
id_descuento	bigint	FK / Nulo → catalogo.descuento	-	Descuento aplicado, si corresponde.
id_aumento_costo	bigint	FK / Nulo → catalogo.aumento_costos	-	Temporada aplicada, si corresponde.
concepto	nvarchar(255)	No Nulo	-	Descripción de la línea.
precio_unitario	decimal(10,2)	No Nulo	Check (>= 0)	Precio por noche o por unidad.
cantidad	int	No Nulo	Check (>= 1)	Noches o unidades cobradas.
subtotal	decimal(10,2)	No Nulo	Check (>= 0)	Calculado: precio_unitario × cantidad.
monto_descuento	decimal(10,2)	No Nulo	Default (0), Check (>= 0)	Monto descontado.
monto_aumento	decimal(10,2)	No Nulo	Default (0), Check (>= 0)	Monto añadido por temporada.
precio_total	decimal(10,2)	No Nulo	Check (>= 0)	Calculado: (subtotal + monto_aumento) - monto_descuento.

Nota: Restricción ck_origen_cobro: exactamente uno entre id_servicio e id_habitacion debe tener valor; cada línea es un cobro de servicio o de hospedaje, nunca ambos ni ninguno.

3.18. Tabla: calidad.resenia

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_resenia	bigint	PK / No Nulo	Identity	Identificador único de la reseña.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía reseñada.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped que escribe la reseña.
calificacion	decimal(2,1)	Nulo	Check (entre 1 y 5)	Calificación otorgada.
comentario	nvarchar(max)	Nulo	-	Comentario del huésped.

Nota: Restricción Unique compuesta: (id_estadia, id_huesped). Además, el trigger trg_actualizar_calificacion impide una segunda reseña para la misma estadía y recalcula la calificación del hotel.

3.19. Notas de diseño

- **Llaves primarias:** todas son `bigint` con Identity y no se actualizan manualmente.
- **Llaves foráneas:** todas usan `ON UPDATE NO ACTION` y `ON DELETE NO ACTION`. SQL Server rechaza `ON UPDATE CASCADE` en este modelo por múltiples rutas de cascada (p. ej., `huesped` alcanza la factura directamente y a través de `reservacion` y `estadia`); como las llaves nunca se modifican, el comportamiento real no cambia.
- **Esquemas:** la separación en `catalogo`, `operacion`, `facturacion`, `calidad` y `reportes` permite otorgar permisos por esquema a cada rol.
- **Lógica de negocio:** una función (`operacion.fn_buscar_habitaciones_disponibles`), un procedimiento (`facturacion.sp_calcular_total_factura`) y cuatro triggers (`tg_check_nivel_habitacion`, `trg_verificar_reserva`, `trg_checkout_factura`, `trg_actualizar_calificacion`).

4. Políticas de seguridad implementadas

La seguridad de CÉNIT se diseñó bajo el principio de **menor privilegio**: cada persona accede únicamente a los datos que necesita para su función. Se apoya en cuatro mecanismos: (1) segmentación de objetos en esquemas, (2) roles de base de datos con permisos otorgados por esquema, (3) denegación explícita (DENY) para el rol de auditoría y (4) SQL Server Audit sobre las tablas financieras.

4.1. Esquemas y justificación

Esquema	Contenido	Justificación
catalogo	hotel, servicio, tipo_habitacion, tipo_comodidad, tipo_empleado, descuento, aumento_costos, comodidad_tipo_habitacion	Aísla reglas de negocio, catálogos y tarifas. La mayoría de los roles solo lee (SELECT); la escritura se reserva a personal gerencial.
operacion	huesped, empleado, habitacion, reservacion, detalle_reservacion, estadia, consumo_servicio	Núcleo transaccional del hotel; concentra el mayor volumen de lecturas, inserciones y actualizaciones.
facturacion	factura, detalle_factura	Datos financieros sensibles y trazabilidad del cobro; permisos restringidos al personal de caja y gerencia.
calidad	resenia	Separa las opiniones de clientes del núcleo transaccional, evitando que análisis de satisfacción compitan por recursos en alta ocupación.
reportes	13 vistas analíticas (v_ingresos_mes, v_tasa_ocupacion_mensual, v_resumen_hotel, v_factura_completa, v_gasto_historica, v_habitaciones_disponibles, etc.)	Capa de inteligencia de negocio: permite consultar sin otorgar acceso directo a las tablas transaccionales.

```
CREATE SCHEMA catalogo;     CREATE SCHEMA operacion;
CREATE SCHEMA facturacion;  CREATE SCHEMA calidad;   CREATE SCHEMA reportes;

-- Traslado de objetos desde dbo (ejemplo; se aplicó a las 18 tablas y 13 vistas)
ALTER SCHEMA facturacion TRANSFER dbo.factura;
ALTER SCHEMA reportes    TRANSFER dbo.v_ingresos_mes;

-- Verificación de la asignación de esquemas
SELECT s.name AS Esquema, t.name AS NombreTabla
FROM sys.tables t JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name IN ('calidad','catalogo','operacion','facturacion','reportes')
ORDER BY s.name, t.name;
```

4.2. Roles, usuarios y permisos

Se definieron siete roles de base de datos. Cada rol tiene asociado un login de servidor y un usuario de base de datos. Por confidencialidad, las contraseñas no se reproducen en este documento.

Rol	Usuario / login	Responsabilidad y permisos
rol_programador (Programador IT)	usr_Rodas_Rodriguez / login_dev	Ciclo de vida del software y optimización. DDL (ALTER, CREATE TABLE/VIEW/PROCEDURE/FUNCTION) sobre todos los esquemas; DML en catalogo, operacion, calidad y reportes; solo SELECT en facturacion; EXECUTE, VIEW DEFINITION, SHOWPLAN, BACKUP DATABASE y BACKUP LOG.
rol_gerente (Gerente)	usr_Alexander_Vazquez / login_gerente	Supervisión global y tarifas. SELECT, INSERT, UPDATE y DELETE en catalogo, operacion, facturacion y calidad; solo SELECT en reportes; EXECUTE de rutinas. Sin DDL.
rol_recepcionista (Recepcionista)	usr_Kenia_Abigail / login_recepcion	Atención de mostrador. SELECT en catalogo; SELECT, INSERT, UPDATE en huesped, reservacion, detalle_reservacion y estadia; SELECT en habitacion y consumo_servicio; SELECT e INSERT en facturacion y calidad; vistas operativas de reportes.
rol_cajero (Cajero)	usr_Cesar_Ramirez / login_cajero	Caja y cobros. SELECT en servicio, descuento, aumento_costos, huesped, estadia y consumo_servicio; SELECT, INSERT, UPDATE en facturacion; vistas financieras. Sin acceso a calidad.
rol_auditor (Auditor)	usr_Oscar_Olivo / login_auditor	Control interno. SELECT en todos los esquemas y VIEW DEFINITION; DENY de INSERT, UPDATE y DELETE para garantizar la inmutabilidad de los datos.
rol_mesero_servicio (Mesero / Servicios)	usr_Bryan_Ortiz / login_mesero (y usr_Sofia_Guardado, supervisora)	Puntos de venta. SELECT en servicio, habitacion y estadia; SELECT e INSERT en consumo_servicio. Sin acceso a facturacion, calidad ni reportes.
rol_atencion_cliente (Atención al Cliente)	usr_Steven_Roque / login_atencion	Satisfacción post-estancia. SELECT en hotel, tipo_habitacion, huesped y estadia; SELECT, INSERT y UPDATE en calidad.resenia (sin DELETE para resguardar la evidencia); vistas cualitativas.

Matriz resumen de permisos por esquema

Rol	catalogo	operacion	facturacion	calidad	reportes
rol_programador	ALL (DDL/DML)	ALL (DDL/DML)	DDL / SELECT	ALL (DDL/DML)	ALL (DDL/DML)
rol_gerente	DML completo	DML completo	DML completo	DML completo	SELECT
rol_recepcionista	SELECT	SELECT, INSERT, UPDATE	SELECT, INSERT	SELECT, INSERT	SELECT (operativo)
rol_cajero	SELECT	SELECT	SELECT, INSERT, UPDATE	Sin acceso	SELECT (financiero)
rol_auditor	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)
rol_mesero_servicio	SELECT (servicio)	SELECT, INSERT (consumos)	Sin acceso	Sin acceso	Sin acceso
rol_atencion_cliente	SELECT (general)	SELECT (huesped/estadia)	Sin acceso	SELECT, INSERT, UPDATE	SELECT (cualitativo)

4.3. Auditoría con SQL Server Audit

Para garantizar la trazabilidad de los datos financieros se configuró una auditoría a nivel de servidor (`Audit_Cenit_Servidor`) con destino a archivo (`/audit/Audit_Cenit`, máximo 100 MB por archivo y 5 archivos de rotación, `ON_FAILURE = CONTINUE`) y una especificación de auditoría de base de datos (`AuditSpec_Facturacion_Sensible`) que registra los **INSERT y UPDATE** ejecutados por cualquier usuario (`public`) sobre `facturacion.factura` y `facturacion.detalle_factura`. Los eventos se consultan con `sys.fn_get_audit_file`, que muestra fecha (UTC), login, usuario, acción, objeto y sentencia ejecutada (Figura 9).

```
CREATE SERVER AUDIT Audit_Cenit_Servidor
  TO FILE (FILEPATH = '/audit/Audit_Cenit', MAXSIZE = 100 MB, MAX_ROLLOVER_FILES = 5)
  WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT Audit_Cenit_Servidor WITH (STATE = ON);

CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Facturacion_Sensible
  FOR SERVER AUDIT Audit_Cenit_Servidor
  ADD (INSERT ON facturacion.factura BY public),
  ADD (UPDATE ON facturacion.factura BY public),
  ADD (INSERT ON facturacion.detalle_factura BY public),
  ADD (UPDATE ON facturacion.detalle_factura BY public)
  WITH (STATE = ON);
```

4.4. Evidencia de pruebas de roles y permisos

Cada rol se probó con `EXECUTE AS USER`, verificando una operación permitida ([ÉXITO]) y una operación que debe ser rechazada ([FALLO ESPERADO]). Todas las pruebas se comportaron según la matriz de permisos.

Prueba	Usuario (rol)	Operación permitida	Operación denegada
1	usr_Rodas_Rodriguez (programador)	CREATE/DROP TABLE en catalogo; SELECT en huesped	DELETE en huesped → Msg 229
2	usr_Alexander_Vazquez (gerente)	SELECT en v_factura_completa y resenia	CREATE TABLE → Msg 262
3	usr_Kenia_Abigail (recepcionista)	SELECT en v_habitaciones_disponibles y huesped	UPDATE habitacion y UPDATE factura → Msg 229
4	usr_Cesar_Ramirez (cajero)	UPDATE de metodo_pago en factura (1 fila)	SELECT en calidad.resenia → Msg 229
5	usr_Oscar_Olivo (auditor)	SELECT en hotel, factura y v_ingresos_mes	INSERT en resenia y DELETE en servicio → Msg 229

```sql
-- ============================================================================
-- PRUEBA 1: Prueba de Éxito: Gestión estructural (DDL) y lectura de datos en esquemas.
-- Prueba de Error: Intento de modificación/borrado de datos (DML) o elevación a nivel de servidor.
-- ============================================================================
EXECUTE AS USER = 'usr_Rodas_Rodriguez';

-- [ÉXITO] Crear y consultar un objeto DDL estructural
CREATE TABLE catalogo.prueba_desarrollo (
    id INT IDENTITY PRIMARY KEY,
    nombre VARCHAR(50)
);
SELECT TOP 2 * FROM operacion.huesped;

-- Limpieza
DROP TABLE catalogo.prueba_desarrollo;

-- [FALLO ESPERADO] Intentar borrar registros (DML no autorizado, solo tiene SELECT y ALTER)
-- Error: Msg 229 - The DELETE permission was denied on the object 'huesped'
DELETE FROM operacion.huesped WHERE id_huesped = 1;

REVERT;
GO
```

103 % ⊗ 17 ⚠ 0

Mensajes

```
Mens. 229, Nivel 14, Estado 5, Línea 113
The DELETE permission was denied on the object 'huesped', database 'cenitDB', schema 'operacion'.

Hora de finalización: 2026-11-04T13:56:04.8852584-06:00
```

Figura 3. Prueba 1 – Programador IT: DDL permitido y DELETE de datos denegado.

```sql
-- ============================================================================
-- PRUEBA 2: Prueba de Éxito: Control operacional transversal DML y lectura de reportes analíticos..
-- Prueba de Error: Creación de objetos estructurales (carece de permisos DDL)
-- ============================================================================
EXECUTE AS USER = 'usr_Alexander_Vazquez';

-- [ÉXITO] Consulta de reportes gerenciales y actualización operativa
SELECT TOP 3 * FROM reportes.v_factura_completa;
SELECT TOP 3 * FROM calidad.resenia;

-- [FALLO ESPERADO] El gerente administra datos, pero no la estructura de la BD
-- Error: Msg 262 - CREATE TABLE permission denied in database 'cenitDB'
CREATE TABLE catalogo.nueva_sucursal (id INT);

REVERT;
GO
-- ============================================================================
```

103 % ⊗ 17 ⚠ 0

Mensajes

```
Mens. 262, Nivel 14, Estado 1, Línea 130
CREATE TABLE permission denied in database 'cenitDB'.

Hora de finalización: 2026-11-04T13:56:39.9919183-06:00
```

Figura 4. Prueba 2 – Gerente: administra datos, pero CREATE TABLE es denegado (sin DDL).

```sql
-- ============================================================================
-- PRUEBA 3: Prueba de Éxito: Consulta de disponibilidad de habitaciones y gestión de huéspedes/estancias.
-- Prueba de Error: Intentos de cobranza directa (facturación) o creación de habitaciones físicas.
-- ============================================================================
GO

EXECUTE AS USER = 'usr_Kenia_Abigail';

-- [ÉXITO] Consulta de disponibilidad física y datos de clientes
SELECT TOP 3 * FROM reportes.v_habitaciones_disponibles;
SELECT TOP 3 * FROM operacion.huesped;

-- [FALLO ESPERADO 1] Alterar inventario físico de habitaciones (solo lectura en habitacion)
-- Error: Msg 229 - The UPDATE permission was denied on the object 'habitacion'
UPDATE operacion.habitacion SET id_tipo_habitacion = 1 WHERE id_habitacion = 1;

-- [FALLO ESPERADO 2] Cobrar o modificar facturas (exclusivo de rol_cajero)
-- Error: Msg 229 - The UPDATE permission was denied on the object 'factura'
UPDATE facturacion.factura SET metodo_pago = 'EFECTIVO' WHERE id_factura = 1;
```

103 % ⊗ 17 ⚠ 0

Resultados | Mensajes

	id_habitacion	hotel	direccion	nivel	numero_habitacion	tipo_habitacion	precio	estado	capacidad_maxima
1	13	Mirador San Salvador	Colonia Escalón, San Salvador	10	3013	Doble Estándar	646.25	DISPONIBLE	7
2	14	Posada del Volcán	Parque Nacional El Boquerón	6	6614	Habitación Accesible (Ley ADA)	285.02	DISPONIBLE	7
3	17	Posada del Volcán	Parque Nacional El Boquerón	9	6917	Suite Ejecutiva	439.08	DISPONIBLE	5

	id_huesped	nombre	correo	telefono	documento	tipo_documento
1	2	Rikki	rtrevance1@over-blog.com	+503 7321-7439	88144014-0	DUI
2	3	Phyllys	pklus2@mysql.com	+503 2722-4817	59864233-2	DUI
3	4	Margery	mtrusdale3@shutterfly.com	+503 7969-8755	81532540-3	DUI

Figura 5. Prueba 3 – Recepcionista: consulta disponibilidad y huéspedes; no puede alterar habitaciones ni facturas.

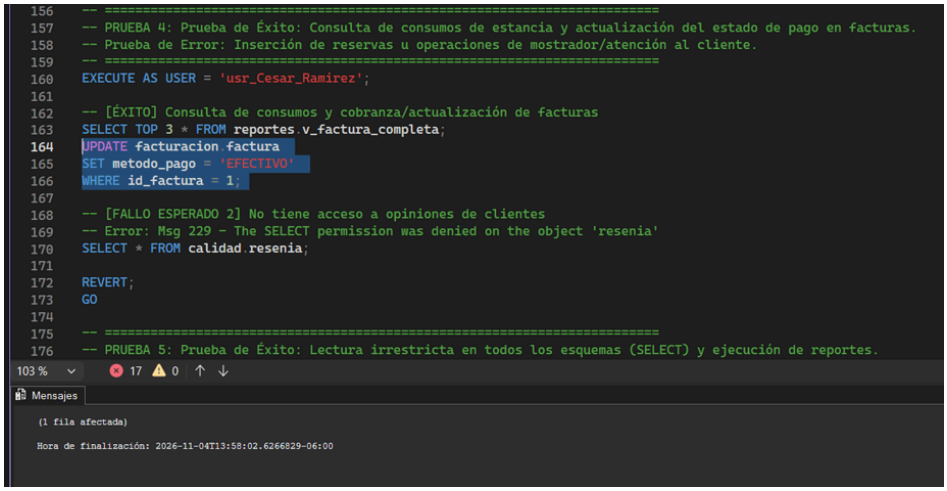


Figura 6. Prueba 4 – Cajero: actualiza el método de pago de una factura (1 fila afectada).

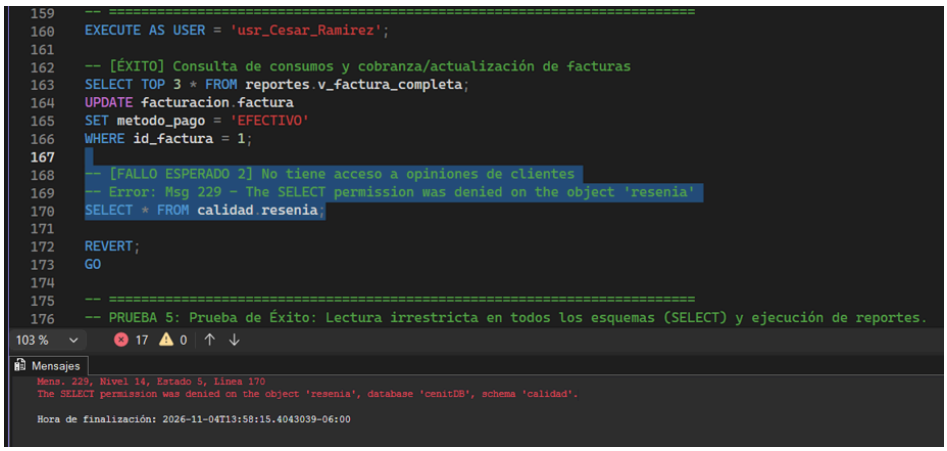


Figura 7. Prueba 4 (cont.) – Cajero: acceso denegado a calidad.resenia.

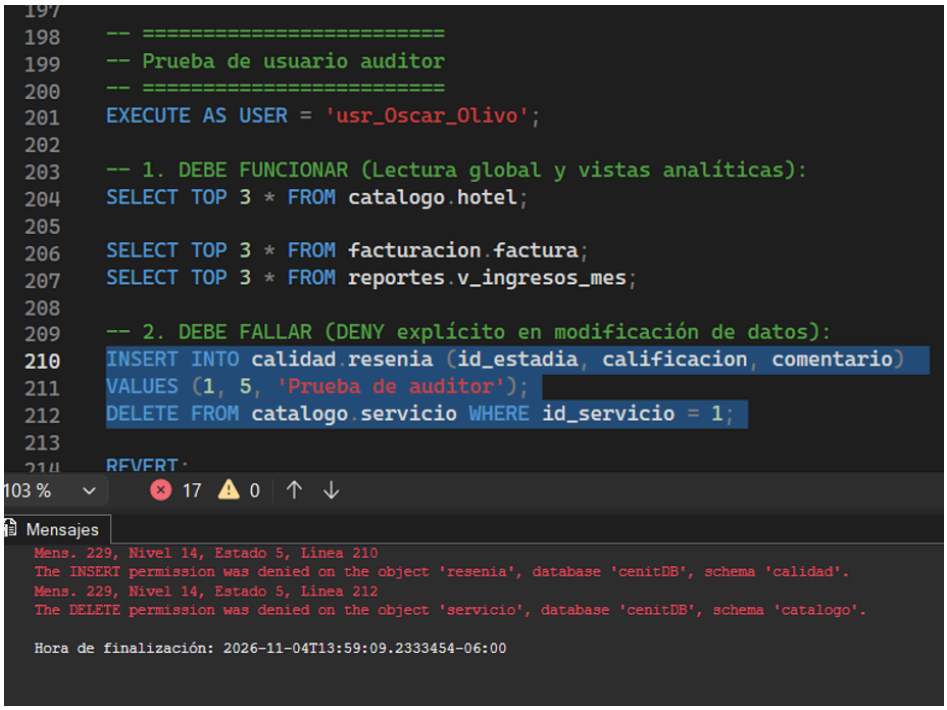


Figura 8. Prueba 5 – Auditor: lectura global permitida; INSERT y DELETE denegados por DENY explícito.

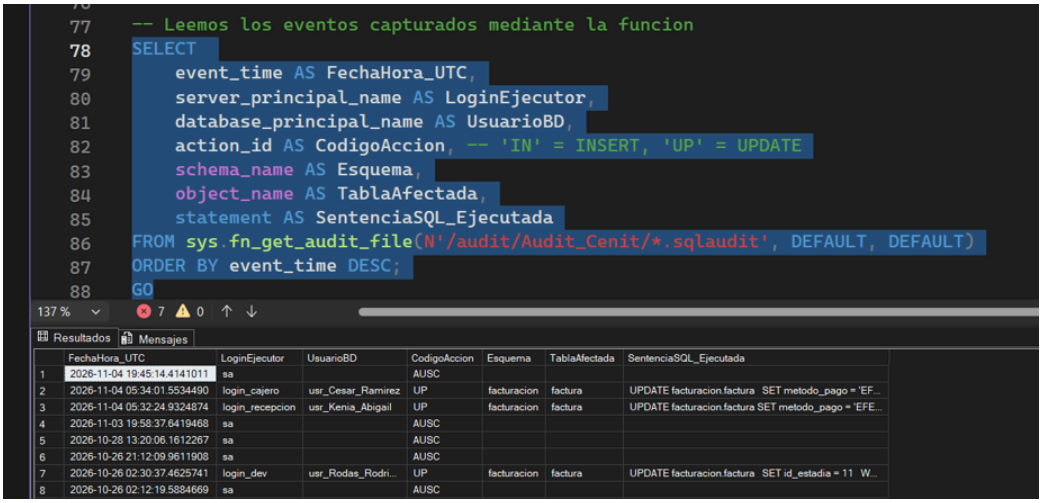


Figura 9. Registro de SQL Server Audit: UPDATE sobre facturacion.factura por login_cajero, login_recepcion y login_dev.

Observaciones y mejoras recomendadas

- **Contraseñas:** en el entorno académico los logins se crearon con `CHECK_POLICY = OFF`. En producción debe activarse `CHECK_POLICY` y `CHECK_EXPIRATION` y rotar las credenciales.
- **TRUSTWORTHY:** el script de auditoría activa `TRUSTWORTHY ON` para ejecutar módulos con `EXECUTE AS`; en producción conviene firmar los módulos con certificado y mantener esta opción desactivada.
- **Protección de datos:** como mejora futura, cifrar los respaldos y aplicar TDE sobre la base de datos.

5. Optimización de consultas e índices

5.1. Metodología

Se optimizaron tres consultas analíticas de facturación, implementadas como vistas con **funciones ventana** (`DENSE_RANK`, `RANK`, `LAG`, `ROW_NUMBER`, `SUM() OVER`). Como la base real contiene solo ~190 facturas y ~1,400 líneas de detalle (un escaneo completo es lo más barato y ningún índice mostraría mejora), se generó una **carga sintética de ~200,000 facturas** (1 a 4 líneas cada una, 75 % de habitación y 25 % de servicio) con el script `00 carga sintetica hoteldb.sql`, que permite revertir los datos al terminar.

El procedimiento fue: (1) crear las vistas base, (2) medir con `SET STATISTICS IO ON` y `SET STATISTICS TIME ON` **sin índices**, (3) crear los índices y vistas indexadas, (4) repetir las mismas consultas y comparar. La métrica principal son las **lecturas lógicas** (páginas de 8 KB leídas), por ser estable entre ejecuciones; el tiempo de CPU se reporta como complemento.

5.2. Índices y objetos aplicados

Objeto	Tipo	Definición	Propósito
IX_factura_huesped	No agrupado	factura (id_huesped) INCLUDE (total_a_pagar)	Agrupar el gasto por huésped sin tocar la tabla base
IX_factura_fecha	No agrupado	factura (fecha) INCLUDE (total_a_pagar)	Index seek por rango de fechas (consulta 2)
IX_detfac_habitacion	No agrupado filtrado	detalle_factura (id_habitacion) INCLUDE (precio_total) WHERE id_habitacion IS NOT NULL	Ingreso por habitación; excluye las líneas de servicio
IX_detfac_factura	No agrupado filtrado	detalle_factura (id_factura) INCLUDE (id_habitacion, precio_total) WHERE id_habitacion IS NOT NULL	Traer solo las líneas de las facturas del rango
ivw_gasto_huesped	Vista indexada	UNIQUE CLUSTERED (id_huesped)	Agregado precalculado: 1 fila por huésped (consulta 1)
ivw_monto_hotel_factura	Vista indexada	UNIQUE CLUSTERED (id_factura, id_hotel)	Monto por factura y hotel precalculado (soporte consulta 2)

Objeto	Tipo	Definición	Propósito
ivw_ingreso_habitacion	Vista indexada	UNIQUE CLUSTERED (id_habitacion)	1 fila por habitación, sin importar las líneas facturadas (consulta 3)
IX_ivw_ingreso_habitacion_top	No agrupado sobre vista	(id_hotel, ingresos DESC, id_habitacion) INCLUDE (...)	Mismo orden del ROW_NUMBER: el top 3 por hotel lee pocas filas

```
CREATE VIEW facturacion.ivw_gasto_huesped WITH SCHEMABINDING AS
SELECT f.id_huesped, SUM(f.total_a_pagar) AS gasto_total, COUNT_BIG(*) AS num_facturas
FROM facturacion.factura AS f GROUP BY f.id_huesped;
GO
CREATE UNIQUE CLUSTERED INDEX CIX_ivw_gasto_huesped
  ON facturacion.ivw_gasto_huesped (id_huesped);
```

Las vistas indexadas requieren `WITH SCHEMABINDING` y `COUNT_BIG(*)`. El optimizador las usa de forma automática en las ediciones Enterprise y Developer; en otras ediciones se invocan con el hint `WITH (NOEXPAND)`.

5.3. Consulta 1 – Ranking de gasto por huésped

Vista: `facturacion.vw_ranking_gastos_huespedes`. Calcula el gasto total por huésped con `DENSE_RANK()` global, `RANK()` por tipo de documento y el porcentaje sobre el total con `SUM() OVER ()`.

Problema (sin índice): agrega todas las facturas en cada ejecución: escaneo completo de `factura` (1,938 páginas).

Solución aplicada: vista indexada `ivw_gasto_huesped` (1 fila por huésped) respaldada por `IX_factura_huesped`; el optimizador lee la vista en lugar de la tabla.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas de la tabla principal	factura: 1,938	ivw_gasto_huesped: 6	99.7 %
Lecturas lógicas totales (incl. huesped: 26)	1,964	32	98.4 %
CPU en ejecución	21 ms	5 ms	76 %
Tiempo transcurrido	24 ms	5 ms	79 %

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 6 ms, elapsed time = 6 ms.
Table 'factura'. Scan count 1, logical reads 1938, physical reads 0, page server rea
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reac
Table 'huesped'. Scan count 1, logical reads 26, physical reads 0, page server reads

 SQL Server Execution Times:
   CPU time = 21 ms,  elapsed time = 24 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-11-04T15:02:15.7612592-06:00
```

Figura 10a. Consulta 1 sin índice (STATISTICS IO/TIME).

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 9 ms, elapsed time = 9 ms.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads
Table 'ivw_gasto_huesped'. Scan count 1, logical reads 6, physical reads 0, page ser
Table 'huesped'. Scan count 1, logical reads 26, physical reads 0, page server reads

 SQL Server Execution Times:
   CPU time = 5 ms,  elapsed time = 5 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-11-04T14:57:24.4352625-06:00
```

Figura 10b. Consulta 1 con vista indexada.

5.4. Consulta 2 – Variación mensual de ingresos por hotel

Vista: `facturacion.vw_variacion_mensual_ingresos_hotel`. Reparte el total de cada factura entre los hoteles según sus líneas de habitación y compara cada mes con el anterior mediante `LAG()`.

Problema (sin índice): el filtro por mes se aplica después de la ventana, por lo que la vista lee todas las facturas y líneas aunque se pida un rango de 3 meses (detalle_factura: 4,736 páginas; factura: 2,038).

Solución aplicada: función con valores de tabla `fn_variacion_mensual_ingresos_hotel(@desde, @hasta)` que filtra las facturas antes de la ventana (incluye un mes previo para el `LAG`), apoyada en `IX_factura_fecha` (seek) e `IX_detfac_factura`. Se verificó con `EXCEPT` que devuelve el mismo resultado que la vista.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas totales	6,794	618	90.9 %
detalle_factura	4,736 (13 escaneos)	324 (94 búsquedas)	93.2 %
factura	2,038	8	99.6 %
CPU en ejecución	620 ms	4 ms	99.4 %
Tiempo transcurrido	105 ms	4 ms	96.2 %

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 20 ms, elapsed time = 21 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
Table 'detalle_factura'. Scan count 13, logical reads 4736, physical reads 0, page s
Table 'habitacion'. Scan count 5, logical reads 16, physical reads 0, page server re
Table 'factura'. Scan count 13, logical reads 2038, physical reads 0, page server re
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server read
Table 'hotel'. Scan count 5, logical reads 4, physical reads 0, page server reads 0,

 SQL Server Execution Times:
   CPU time = 620 ms,  elapsed time = 105 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
```

Figura 11a. Consulta 2 sin índice (rango 2026-03 a 2026-06).

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 16 ms, elapsed time = 16 ms.
Table 'factura'. Scan count 2, logical reads 8, physical reads 0, page server reads
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reac
Table 'hotel'. Scan count 0, logical reads 20, physical reads 0, page server reads (
Table 'habitacion'. Scan count 0, logical reads 266, physical reads 0, page server r
Table 'detalle_factura'. Scan count 94, logical reads 324, physical reads 0, page se

 SQL Server Execution Times:
   CPU time = 4 ms,  elapsed time = 4 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-11-04T14:57:53.8763845-06:00
```

Figura 11b. Consulta 2 con función filtrada e índices.

5.5. Consulta 3 – Top 3 habitaciones más rentables por hotel

Vista: `facturacion.vw_top3_habitaciones_rentables`. Suma los ingresos por habitación y obtiene las tres primeras de cada hotel con `ROW_NUMBER() OVER (PARTITION BY id_hotel ORDER BY ingresos DESC)`.

Problema (sin índice): lee todas las líneas de habitación de `detalle_factura` (4,504 páginas) para devolver ~9 filas.

Solución aplicada: vista indexada `ivw_ingreso_habitacion` (1 fila por habitación) con el índice `IX_ivw_ingreso_habitacion_top`, cuyo orden coincide con el de la ventana.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas de la tabla principal	detalle_factura: 4,504	ivw_ingreso_habitacion: 10	99.8 %
Lecturas lógicas totales	4,597	59	98.7 %
CPU en ejecución	22 ms	2 ms	91 %
Tiempo transcurrido	22 ms	1 ms	95 %

```
 SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.

  SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
 SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.
 SQL Server parse and compile time:
    CPU time = 11 ms, elapsed time = 11 ms.
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads
 Table 'detalle_factura'. Scan count 1, logical reads 4504, physical reads 0, page ser
 Table 'habitacion'. Scan count 1, logical reads 44, physical reads 0, page server rea
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads
 Table 'tipo_habitacion'. Scan count 1, logical reads 25, physical reads 0, page serve
 Table 'hotel'. Scan count 0, logical reads 24, physical reads 0, page server reads 0,

  SQL Server Execution Times:
    CPU time = 22 ms,  elapsed time = 22 ms.
 SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.

  SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Figura 12a. Consulta 3 sin índice.

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 11 ms, elapsed time = 11 ms.
Table 'ivw_ingreso_habitacion'. Scan count 1, logical reads 10, physical reads 0, pa
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server read
Table 'tipo_habitacion'. Scan count 1, logical reads 25, physical reads 0, page serv
Table 'hotel'. Scan count 0, logical reads 24, physical reads 0, page server reads 0

 SQL Server Execution Times:
   CPU time = 2 ms,  elapsed time = 1 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-11-04T14:58:33.3981223-06:00
```

Figura 12b. Consulta 3 con vista indexada.

5.6 Planes de ejecución

Para entender, comparar y explicar cada plan.

Resumen Comparativo

Plan	Costo	Tiempo	CPU	Memoria concedida	Lecturas lógicas (apro
C1 sin índice	**1.502**	**41 ms**	**41 ms**	**4.6 MB**	**1,519**
C1 optimizado	**0.072**	**4 ms**	**4 ms**	**4.6 MB**	**32**
C2 sin índice	**6.021**	**89 ms**	**585 ms**	**187 MB (DOP 12)**	**6,728**
C2 optimizado	**0.474**	**2 ms**	**2 ms**	**6.4 MB (DOP 1)**	**544**

C3 sin índice	**4.411**	**23 ms**	**22 ms**	**137 MB (usó 2.7 MB)**	**4,596**
C3 optimizado	**0.024**	**< 1 ms**	**0 ms**	**2.2 MB**	**59**

NOTA: La base es pequeña (~200 mil filas de facturas), por lo que los milisegundos absolutos son bajos. Lo importante es que las lecturas y la memoria crecen con los datos: con millones de filas, el plan sin índice se degradaría mucho más que el optimizado.

Caso 1: ranking de gasto de huéspedes

Consulta: SELECT * FROM facturacion.vw_ranking_gastos_huespedes ORDER BY ranking_global, id_huesped

Sin índice

Para saber cuánto gastó cada huésped, el sistema revisa las 200,309 facturas una por una, las suma por huésped y luego arma el ranking. Es como contar todos los recibos del hotel cada vez que alguien pide la lista de mejores clientes.

- Un **Index Scan** sobre IX_factura_huesped lee 200,309 filas (1,493 lecturas lógicas).
- **Stream Aggregate** agrupa por id_huesped y deja 741 filas. Es el cuello de botella: 38 ms de los 41 ms totales.
- Un **Merge Join** une el resultado con huesped (1,172 filas, 26 lecturas).
- Después hay 3 pasos de Sort + Window Aggregate que calculan los rankings (uno particionado por tipo_documento).

Optimizado

Se creó un resumen ya calculado del gasto por huésped. Ahora el sistema lee 741 renglones en vez de 200 mil recibos.

- Hay una **vista indexada** (ivw_gasto_huesped, índice CIX_ivw_gasto_huesped). El Clustered Index Scan sobre ella lee 741 filas con 6 lecturas.
- El agregado desapareció del plan. Los Sort y Window Aggregate siguen, pero trabajan sobre pocos datos ya agregados.
- El Merge Join pasó de 39 ms a 1 ms.

> El costo baja ~95%, las lecturas ~47 veces y el tiempo ~10 veces. El plan sin índice ya usaba un índice no clustered, pero no resolvía el problema: el cuello de botella era calcular la agregación, no encontrar los datos.

Caso 2: variación mensual de ingresos por hotel

Consulta original: SELECT * FROM facturacion.vw_variacion_mensual_ingresos_hotel WHERE mes >= @1 AND mes < @2 ORDER BY hotel, mes (@1 = 2026-03-01, @2 = 2026-06-01)

Sin índice

Se pide la variación de ingresos de marzo a mayo (solo 10 filas). Para responder, el sistema calcula todos los meses de toda la historia de todos los hoteles y recién al final descarta lo que no se pidió.

- **Filtro tardío:** el Filter sobre el mes aparece en el nodo 5, cuando ya se procesaron ~200,340 filas. No puede empujarse hacia abajo porque la vista usa una función de ventana (probablemente LAG, que compara con el mes anterior).

- **Clustered Index Scan** de factura (200,309 filas) y detalle_factura (200,388 filas), con hash joins y hash aggregates.
- **Paralelismo DOP 12:** 585 ms de CPU para 89 ms de reloj.
- **Memoria:** pide 187 MB y usa 84 MB.
- **Waits dominantes:** BPSORT (278 ms) y RESERVED_MEMORY_ALLOCATION (118 ms).
- Estimó 17,429 filas y devolvió 10.

Optimizado

Ahora la consulta va directo a las facturas de febrero a mayo, en vez de recorrerlas todas.

- Se reemplazó la vista por una **función con parámetros de fecha** (fn_variacion_mensual_ingresos_hotel). Es un cambio de diseño, no solo un índice.
- Un Index Seek en IX_factura_fecha lee solo 94 facturas, con rango 2026-02-01 a 2026-06-01. Se incluye febrero para poder calcular el LAG de marzo; un Filter posterior descarta febrero.
- IX_detfac_factura permite Nested Loops con seeks hacia detalle_factura, y se usa pk_habitacion para llegar a la habitación.
- Plan serial (DOP 1), 6 MB de memoria y 2 ms de duración.

Caso 3: Top 3 habitaciones más rentables por hotel

Consulta: SELECT * FROM facturacion.vw_top3_habitaciones_rentables ORDER BY hotel, posicion

Sin índice

Para encontrar las 3 mejores habitaciones por hotel, el sistema suma todos los ingresos de las 200 mil líneas de factura y luego se queda con solo 12 filas.

- **Clustered Index Scan** sobre detalle_factura: 201,758 filas leídas. Un Hash Match Aggregate suma por habitación.
- **Sort + Window Aggregate** (tipo ROW_NUMBER) numera las 1,021 habitaciones; un **Filter** "<= 3" deja solo 12.
- **Excessive Grant:** se reservaron 137 MB y se usaron 2.7 MB. Esto puede hacer esperar a otras consultas.
- Hay dos **Adaptive Join**. Eligieron la rama de Nested Loops con 12 seeks; las ramas de scan (hash) se ejecutaron 0 veces.

Optimizado

Hay una tabla resumen de ingresos por habitación que se mantiene sola. Ya no se suman 200 mil líneas cada vez.

- Una **vista indexada** (ivw_ingreso_habitacion, índice IX_ivw_ingreso_habitacion_top) entrega los ingresos ya agregados con 10 lecturas.
- El Window Aggregate y el Filter "<= 3" se mantienen; los joins pasaron a Nested Loops livianos.
- La memoria baja de 137 MB a 2.2 MB y las lecturas de ~4,600 a ~59.

> **Observación:** aun así, se leen las 1,021 filas del índice para devolver 12. Se podría mejorar más, pero a esta escala ya es irrelevante.

5.7. Resumen y conclusiones

Consulta	Técnica analítica	Lecturas lógicas (sin → con)	Reducción	CPU (sin → con)
1. Ranking de huéspedes	DENSE_RANK, RANK, SUM OVER	1,964 → 32	98.4 %	21 → 5 ms
2. Variación mensual	LAG	6,794 → 618	90.9 %	620 → 4 ms
3. Top 3 habitaciones	ROW_NUMBER	4,597 → 59	98.7 %	22 → 2 ms

- Con volumen realista (~200,000 facturas), los índices de cobertura y las vistas indexadas reducen entre 91 % y 99 % las lecturas lógicas de las tres consultas.
- Los tiempos en milisegundos varían entre ejecuciones; por eso la comparación se basa en lecturas lógicas.
- Costo: cada índice y vista indexada añade espacio y trabajo en las escrituras (INSERT/UPDATE de facturas). Este costo se considera en el dimensionamiento (sección 6), y `SCHEMABINDING` impide modificar las tablas base sin antes eliminar la vista.

6. Estrategia de dimensionamiento

6.1. Situación actual y medición

Con los datos reales, la base ocupa **menos de 2 MB**. Las tablas de catálogo con 4 a 10 filas consumen 16 KB de datos (2 páginas de 8 KB) y reservan 72 KB, porque el motor asigna un mínimo de páginas para la estructura de datos y la navegación de índices. En tablas intermedias como `detalle_factura` y `comodidad_tipo_habitacion`, el espacio real duplica o triplica el cálculo puro de datos por el índice agrupado y las llaves foráneas.

Con la carga de prueba de ~200,000 facturas, las lecturas de un escaneo completo (cada página = 8 KB) permiten medir las tablas que dominan el crecimiento:

Tabla	Páginas leídas (escaneo)	Tamaño aproximado	Bytes por factura
facturacion.factura	2,038	≈ 15.9 MB	≈ 82 B
facturacion.detalle_factura (≈ 2.5 líneas por factura)	4,504	≈ 35.2 MB	≈ 180 B
Total tablas de facturación	6,542	≈ 51 MB	≈ 262 B

Para obtener el tamaño real por tabla en cualquier momento se usa:

```
SELECT s.name + '.' + t.name AS tabla,
       SUM(ps.reserved_page_count) * 8 / 1024.0 AS reservado_mb,
       SUM(CASE WHEN ps.index_id IN (0,1) THEN ps.row_count ELSE 0 END) AS filas
FROM sys.dm_db_partition_stats ps
JOIN sys.tables t ON t.object_id = ps.object_id
JOIN sys.schemas s ON s.schema_id = t.schema_id
GROUP BY s.name, t.name ORDER BY reservado_mb DESC;
```

6.2. Proyección de crecimiento

El crecimiento lo domina la facturación. Con el dato medido (≈ 0.26 KB de datos por factura), más ≈ 0.17 KB de índices (estimado para los 4 índices de la sección 5) y ≈ 0.20 KB de tablas operativas asociadas (reservación, estadía, consumos; estimado), se adopta **≈ 0.65 KB por factura**. Los supuestos de volumen son hipótesis de planificación.

Escenario (supuesto)	Facturas por año	Año 1	Año 3	Año 5
Operación media (≈ 100 facturas/día)	36,500	≈ 23 MB	≈ 70 MB	≈ 116 MB
Operación alta (cadena; volumen de la carga de prueba por año)	200,000	≈ 127 MB	≈ 381 MB	≈ 635 MB

6.3. Dimensionamiento recomendado

Recurso	Configuración recomendada	Justificación
Archivo de datos (MDF)	Inicial 256 MB; crecimiento fijo de 128 MB (no porcentual)	Cubre más de 5 años en escenario medio y evita autocrecimientos frecuentes
Archivo de log (LDF)	Inicial 256 MB; crecimiento fijo de 128 MB	En modelo FULL el log crece hasta el siguiente respaldo (cada 15 min); las cargas masivas (como las 200,000 facturas de prueba) generan un log de varias decenas de MB
Disco /backup	Mínimo 10 GB, en un volumen distinto al de datos y log	Los Full comprimidos pesan una fracción de la base; 4 Full + 7 Diff + 7 días de logs caben con holgura incluso en el escenario alto (≈ 2 GB)
Memoria	Mínimo 2 GB para SQL Server; fijar `max server memory` dejando memoria al sistema operativo	Las vistas indexadas y los índices de cobertura se benefician de datos en caché
Monitoreo	`DBCC SQLPERF(LOGSPACE)` y la consulta de tamaño por tabla; alerta al 80 % de uso	Permite ampliar archivos de forma planificada y no reactiva
Mantenimiento	Actualizar estadísticas y reorganizar (5–30 % de fragmentación) o reconstruir (> 30 %) índices semanalmente	Conserva las mejoras de la sección 5 a medida que crece la base

7. Estrategia de respaldo y recuperación

7.1. Objetivos de recuperación (RPO y RTO)

Objetivo	Valor	Justificación	Cómo se cumple
RPO (Recovery Point Objective)	15 minutos	La base procesa transacciones críticas en tiempo real (facturas, cobros con tarjeta, confirmación de reservaciones, registro de huéspedes). Perder más de 15 minutos implicaría perder pagos y generar sobreventa de habitaciones.	Modelo de recuperación FULL y respaldo del log de transacciones cada 15 minutos
RTO (Recovery Time Objective)	1 hora	Ante una falla crítica (caída del servidor, daño de hardware o corrupción de MDF/LDF) recepción puede operar temporalmente con contingencias manuales; 1 hora permite al DBA aprovisionar el entorno y restaurar la cadena de respaldos.	Respaldos comprimidos y procedimiento de restauración encadenada Full + Diff + Logs documentado y probado

Relación entre RPO, RTO, WRT y MTD

Además del RPO y el RTO, la continuidad del servicio se describe con otras dos métricas: el WRT (Work Recovery Time) y el MTD (Maximum Tolerable Downtime). El esquema siguiente muestra cómo se encadenan en el tiempo a partir de la última copia de seguridad: el RPO indica cuántos datos se pierden, el RTO cuánto tarda en restaurarse la información, el WRT cuánto tarda en normalizarse el servicio y el MTD cuánto puede permanecer inoperativo antes de que las consecuencias sean inaceptables.



Esquema 7.1. Relación temporal entre RPO, RTO, WRT y MTD a partir de una caída del sistema.

Métrica	Qué describe	Aplicación en CÉNIT
RPO (Recovery Point Objective)	Cuántos datos se pueden perder: desde la última copia de seguridad hasta la caída del sistema.	15 minutos: modelo de recuperación FULL y respaldo del log cada 15 minutos.
RTO (Recovery Time Objective)	Cuánto tiempo pasa hasta que los datos están restaurados después de la caída.	≈ 25–30 min estimados: diagnóstico (15), Full y Diferencial (5) y logs posteriores (5–10).
WRT (Work Recovery Time)	Tiempo en el que se recuperará el servicio real mediante la verificación de los sistemas y la puesta en línea, tras haberlos recuperado en la parte del RTO.	≈ 15 min estimados: DBCC CHECKDB y validación de conteos (5) y reconexión de aplicaciones y pruebas funcionales (10).
MTD (Maximum Tolerable Downtime)	Tiempo máximo que puede estar inoperativo el servicio antes de que las consecuencias sean inaceptables para la empresa. Debe cumplirse RTO + WRT ≤ MTD.	1 hora (límite de la sección 7.1). El total estimado de 40–45 min queda dentro de ese límite.

Nota: el valor de 1 hora de la tabla anterior se usa como tope del tiempo total hasta volver a operar (RTO + WRT), es decir, como MTD. Los estimados parciales de la sección 7.4 se reparten entre el RTO (restauración de los datos) y el WRT (verificación y puesta en línea).

Los valores dependen de la naturaleza del servicio: no tiene la misma criticidad un proceso transaccional de producción (reservaciones y facturación) que uno administrativo o analítico (reportes y dashboard), por lo que sus objetivos de recuperación pueden ser distintos. Por ello conviene clasificar los servicios y analizar su impacto en el negocio.

7.2. Esquema de respaldos automatizados

La base opera en modelo de recuperación FULL (`ALTER DATABASE HotelDB SET RECOVERY FULL`). Los respaldos se automatizan con tres jobs de SQL Server Agent (base `msdb`), cuyo archivo de destino lleva fecha y hora (`yyyymmdd_hhmmss`) para no sobrescribir copias anteriores.

Tipo	Job	Frecuencia	Archivo / opciones	Retención recomendada
Completo (Full)	HotelDB_Auto_Backup_Full	Semanal: lunes 00:00	HotelDB_Full_<fecha>.bak · FORMAT, COMPRESSION	4 semanas
Diferencial	HotelDB_Auto_Backup_Diff	Diario 01:00	HotelDB_Diff_<fecha>.bak · FORMAT, COMPRESSION, DIFFERENTIAL	7 días

Tipo	Job	Frecuencia	Archivo / opciones	Retención recomendada
Log de transacciones	HotelDB_Auto_Backup_Log	Cada 15 minutos, todo el día	HotelDB_Log_<fecha>.trn · COMPRESSION, NOINIT	7 días

```
EXEC dbo.sp_add_job @job_name = N'HotelDB_Auto_Backup_Log', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'HotelDB_Auto_Backup_Log', @step_name = N'Paso_Backup_Log',
  @subsystem = N'TSQL', @database_name = N'master', @command = N'
  DECLARE @Archivo NVARCHAR(500) = N''/backup/HotelDB_Log_''
    + CONVERT(VARCHAR(8), GETDATE(), 112) + ''_'' + REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108),
'':'', '''') + ''.trn'';
  BACKUP LOG [HotelDB] TO DISK = @Archivo WITH COMPRESSION, NOINIT, STATS = 10;';
EXEC dbo.sp_add_jobschedule @job_name = N'HotelDB_Auto_Backup_Log', @name = N'Cada_15_Minutos',
  @freq_type = 4, @freq_interval = 1, @freq_subday_type = 4, @freq_subday_interval = 15,
  @active_start_time = 000000;
EXEC dbo.sp_add_jobserver @job_name = N'HotelDB_Auto_Backup_Log';
```

Los jobs Full y Diferencial siguen el mismo patrón (Full: `freq_type = 8`, `freq_interval = 2`; Diferencial: `freq_type = 4`, 01:00). Cada job se validó con `sp_start_job`.

7.3. Procedimiento de recuperación

La recuperación se realiza como restauración encadenada: último Full, último Diferencial y todos los respaldos de log posteriores, en orden cronológico. Solo el último paso usa `WITH RECOVERY`; con `STOPAT` se puede recuperar a un instante concreto (por ejemplo, antes de un borrado accidental).

```
USE master;
ALTER DATABASE HotelDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;   -- desconectar sesiones

RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Full_<fecha>.bak'
  WITH NORECOVERY, REPLACE, STATS = 10;
RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Diff_<fecha>.bak'
  WITH NORECOVERY, STATS = 10;
RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_<fecha1>.trn' WITH NORECOVERY;
RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_<fechaN>.trn' WITH RECOVERY;  -- último log
                                                                   -- (STOPAT = '<fecha hora>'
opcional)
ALTER DATABASE HotelDB SET MULTI_USER;

-- Verificación posterior
USE HotelDB;
SELECT 'Huéspedes' AS Tabla, COUNT(*) AS Total FROM operacion.huesped
UNION ALL SELECT 'Reservaciones', COUNT(*) FROM operacion.reservacion
UNION ALL SELECT 'Facturas', COUNT(*) FROM facturacion.factura;
DBCC CHECKDB ('HotelDB');
```

Prueba realizada: el script de validación registra un huésped nuevo antes de los respaldos manuales (Full, Diferencial y Log), restaura la cadena completa y comprueba los conteos de huéspedes, reservaciones y facturas, además de ejecutar `DBCC CHECKDB` para confirmar la integridad.

7.4. Cumplimiento del RTO

Paso	Tiempo estimado
Diagnóstico y aprovisionamiento del servidor o instancia	15 min
Restauración del Full y del Diferencial (base de pocos cientos de MB, respaldos comprimidos)	5 min
Aplicación de los logs posteriores (hasta 96 por día)	5–10 min
`DBCC CHECKDB` y validación de conteos	5 min
Reconexión de aplicaciones y pruebas funcionales	10 min
Total estimado	**≈ 40–45 min (menor a 60 min)**

	Nombre_Job	Paso	Fecha_Hora_Ejecucion	Estado_Ejecucion	Duracion
1	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-10 02:10:15.000	Exitoso	00:00:01
2	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-10 02:10:15.000	Exitoso	00:00:01
3	HotelDB_Auto_Backup_Diff	Paso_Backup_Diff	2026-11-10 01:48:24.000	Exitoso	00:00:00
4	HotelDB_Auto_Backup_Diff	(Job outcome)	2026-11-10 01:48:24.000	Exitoso	00:00:00
5	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 21:00:00.000	Exitoso	00:00:01
6	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 21:00:00.000	Exitoso	00:00:01
7	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 20:45:00.000	Exitoso	00:00:00
8	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 20:45:00.000	Exitoso	00:00:00
9	HotelDB_Auto_Backup_Full	Paso_Backup_Full	2026-11-09 20:40:40.000	Exitoso	00:00:00
10	HotelDB_Auto_Backup_Full	(Job outcome)	2026-11-09 20:40:40.000	Exitoso	00:00:00
11	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 20:30:01.000	Exitoso	00:00:00
12	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 20:30:01.000	Exitoso	00:00:00
13	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 04:45:01.000	Exitoso	00:00:00
14	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 04:45:01.000	Exitoso	00:00:00
15	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 04:30:01.000	Fallido	00:00:00

Figura 13. Jobs de respaldo en SQL Server Agent (HotelDB_Auto_Backup_Full / Diff / Log) y su historial de ejecución.

	Base_Datos	Tipo_Backup	Ruta_Y_Nombre_Archivo	Fecha_Creacion	Tamano_MB
1	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261110_021500.trn	2026-11-10 02:15:00.000	0.08
2	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261110_021015.trn	2026-11-10 02:10:15.000	0.14
3	HotelDB	Respaldo Diferencial	/backup/HotelDB_Diff_20261110_014824.bak	2026-11-10 01:48:24.000	3.09
4	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_210001.trn	2026-11-09 21:00:01.000	0.08
5	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_204500.trn	2026-11-09 20:45:00.000	0.14
6	HotelDB	Respaldo Full (Completo)	/backup/HotelDB_Full_20261109_204040.bak	2026-11-09 20:40:40.000	12.09
7	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_203001.trn	2026-11-09 20:30:01.000	0.08
8	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_044501.trn	2026-11-09 04:45:01.000	0.08
9	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_041500.trn	2026-11-09 04:15:00.000	0.08
10	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_040000.trn	2026-11-09 04:00:00.000	0.08
11	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_034501.trn	2026-11-09 03:45:01.000	0.08
12	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_033001.trn	2026-11-09 03:30:01.000	0.08
13	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_031500.trn	2026-11-09 03:15:00.000	0.08
14	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_030000.trn	2026-11-09 03:00:00.000	0.08
15	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_024501.trn	2026-11-09 02:45:01.000	0.08

Figura 14. Historial de backups Generrados

8. Dashboard en Power BI

8.1. Objetivo y fuente de datos

El dashboard ofrece a la gerencia una vista consolidada del desempeño del hotel: ingresos, ocupación, comportamiento de huéspedes y rentabilidad de habitaciones. Power BI Desktop se conecta a SQL Server (`HotelDB`) y consume las **vistas del esquema** `reportes` y las tres vistas analíticas optimizadas de la sección 5, en lugar de las tablas transaccionales. De esta forma se respeta el modelo de seguridad (acceso de solo lectura a la capa analítica) y se aprovechan los índices ya creados.

8.2. Indicadores y visualizaciones

Vista de origen	Indicador	Visualización
reportes.v_ingresos_mes	Ingresos mensuales	Gráfico de columnas / líneas por mes
reportes.v_tasa_ocupacion_mensual	Tasa de ocupación mensual	Gráfico de líneas con porcentaje
reportes.v_resumen_hotel	Resumen por hotel (habitaciones, calificación)	Tarjetas y tabla por hotel
reportes.v_huespedes_por_hotel	Huéspedes por hotel	Gráfico de barras
reportes.v_servicios_mas_consumidos_tipo_habitacion	Servicios más consumidos por tipo de habitación	Barras apiladas / matriz
facturacion.vw_ranking_gastos_huespedes	Top de huéspedes por gasto	Tabla ordenada con ranking
facturacion.vw_variacion_mensual_ingresos_hotel	Variación mensual de ingresos (%)	Líneas por hotel con segmentación de fechas
facturacion.vw_top3_habitaciones_rentables	Top 3 habitaciones por hotel	Tabla / matriz por hotel

8.3. Conexión, seguridad y actualización

- Conexión mediante un login de solo lectura asociado a un rol con SELECT sobre `reportes` (por ejemplo, el rol del gerente o del auditor); las credenciales no se incrustan en el archivo compartido.
- Modo de importación: el volumen es reducido y las consultas ya están optimizadas; la actualización programada se fija en horas de baja ocupación, posteriores al respaldo diferencial de las 01:00.
- Segmentaciones por hotel y por rango de fechas para filtrar todas las páginas del informe.

8.4. Evidencia del dashboard

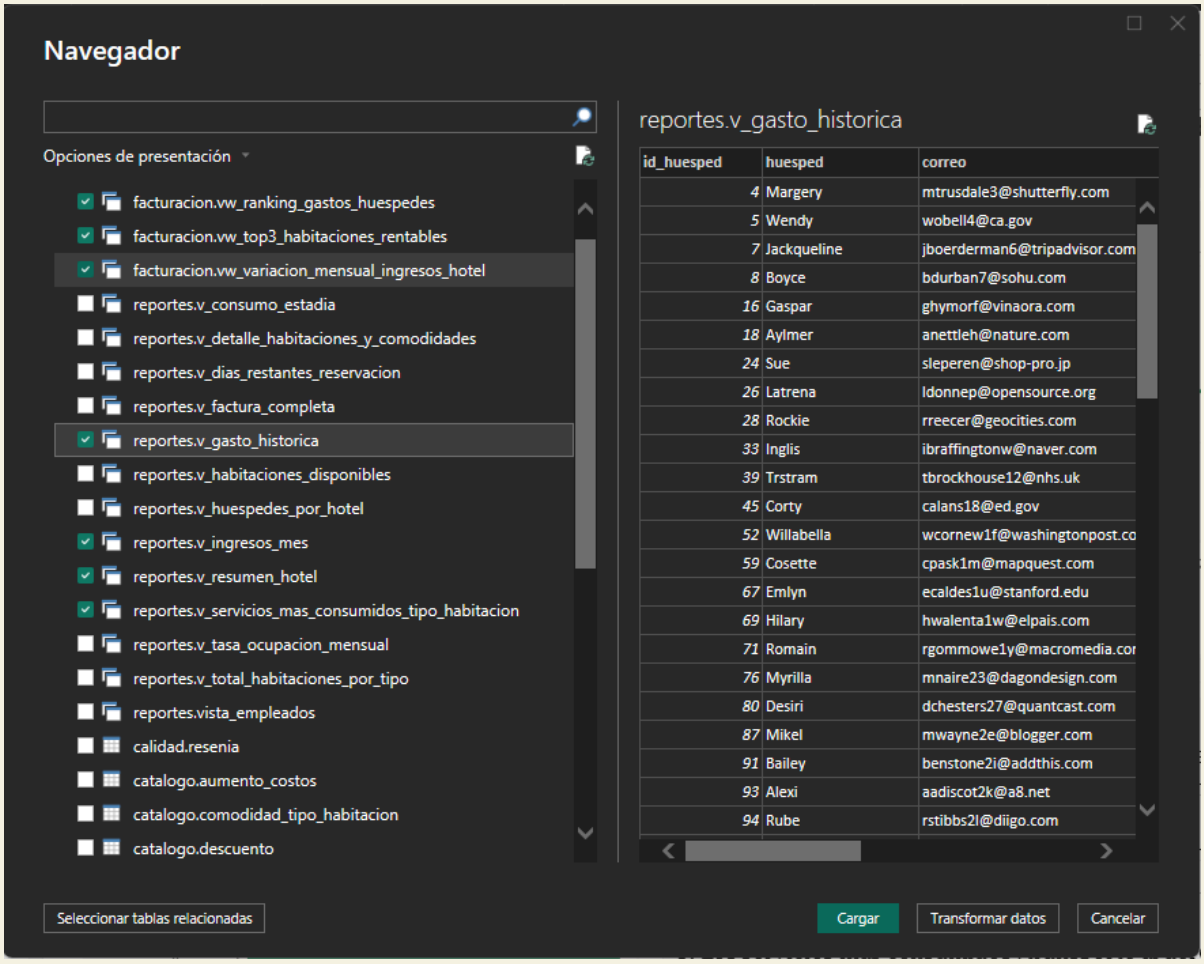




Figura 16. Modelo de datos en Power BI (tablas/vistas importadas y relaciones).



Figura 17. Dashboard – página 1: ingresos, ocupación y variación mensual.

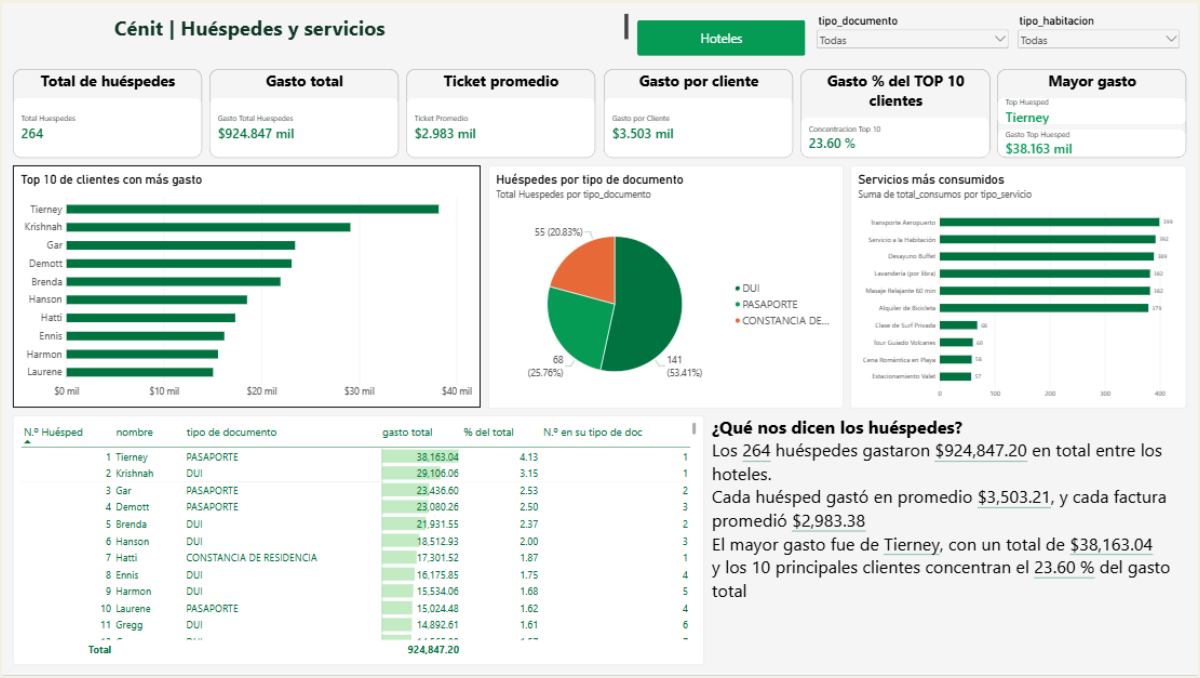


Figura 18. Dashboard – página 2: huéspedes, servicios y habitaciones más rentables.

9. Conclusiones

- El modelo en 3FN, organizado en esquemas, permite aplicar seguridad por esquema y mantener la integridad de las reglas de negocio con restricciones y triggers.
- La seguridad combina roles de menor privilegio, DENY explícito para auditoría y SQL Server Audit sobre facturación, validados con pruebas por rol.
- Los índices de cobertura y las vistas indexadas redujeron entre 91 % y 99 % las lecturas lógicas de las consultas analíticas con ~200,000 facturas.
- El dimensionamiento proyecta menos de 650 MB en cinco años aun en el escenario alto, y los respaldos Full, Diferencial y Log automatizados sustentan un RPO de 15 minutos y un RTO de 1 hora.
- El dashboard en Power BI consume la capa analítica sin exponer las tablas transaccionales.

Anexo A. Calendario de políticas de backup – HotelDB

Calendario de planificación elaborado por el equipo; complementa la sección 7.

Elemento	Valor
Base de datos y modelo	HotelDB · modelo de recuperación FULL
Objetivos de recuperación	RPO ≤ 15 min · RTO < 30 min (meta operativa del calendario)
Automatización	3 jobs activos de SQL Server Agent (Full / Diff / Log)
Volumen semanal estimado	680 backups por semana (1 Full, 7 Diff, 672 Log)

Nota: la sección 7.1 establece un RTO de 1 hora como límite aceptable; el calendario maneja una meta operativa más exigente (< 30 min).

A.1. Políticas de backup y automatización de jobs

Tipo	Job SQL Agent	Frecuencia / hora	Ruta y patrón de archivo	Parámetros	Propósito y RPO
Full (completo)	HotelDB_Auto_Backup_Full	Semanal: lunes 00:00	/backup/HotelDB_Full_YYYYMMDD_HHMMSS.bak	FORMAT, COMPRESSION	Línea base integral del sistema

Tipo	Job SQL Agent	Frecuencia / hora	Ruta y patrón de archivo	Parámetros	Propósito y RPO
Diferencial	HotelDB_Auto_Backup_Diff	Diario (lun–dom) 01:00	/backup/HotelDB_Diff_YYYYMMDD_HHMMSS.bak	FORMAT, COMPRESSION, DIFFERENTIAL	Cambios acumulados desde el último Full
Log de transacciones	HotelDB_Auto_Backup_Log	Continuo cada 15 min, 24/7 (00:00 a 23:45)	/backup/HotelDB_Log_YYYYMMDD_HHMMSS.trn	COMPRESSION, NOINIT	RPO ≤ 15 min; trunca el log inactivo

A.2. Calendario semanal

Franja horaria	Lunes	Martes a domingo
00:00 – 00:30 (ventana crítica)	FULL BACKUP (00:00) + Log (00:00, 00:15)	Log Backup (00:00, 00:15)
00:30 – 01:00 (madrugada temprana)	Log Backup (00:30, 00:45)	Log Backup (00:30, 00:45)
01:00 – 01:30 (ventana diferencial)	DIFERENCIAL (01:00) + Log	DIFERENCIAL (01:00) + Log
01:30 – 06:00 (madrugada valle)	Log Backup cada 15 min (18 ejecuciones)	Log Backup cada 15 min (18 ejecuciones)
06:00 – 12:00 (turno mañana / check-out)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
12:00 – 18:00 (turno tarde / check-in)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
18:00 – 23:45 (turno noche / cierre)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
Total diario (semana: 98 + 6 × 97 = 680)	**98** (1 Full, 1 Diff, 96 Log)	**97** por día (1 Diff, 96 Log)

A.3. Guía de restauración de emergencia

Paso	Archivo requerido	Cláusula	Instrucción T-SQL	Estado de HotelDB
1 · Full	HotelDB_Full_YYYYMMDD_HHMMSS.bak	NORECOVERY, REPLACE	`RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Full_20250217_000000.bak' WITH NORECOVERY, REPLACE;`	RESTORING (inaccesible)
2 · Diferencial	HotelDB_Diff_YYYYMMDD_HHMMSS.bak	NORECOVERY	`RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Diff_20250219_010000.bak' WITH NORECOVERY;`	RESTORING (avanza LSN)
3 · Logs intermedios	HotelDB_Log_YYYYMMDD_HHMMSS.trn	NORECOVERY	`RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_20250219_011500.trn' WITH NORECOVERY;` (repetir en orden cronológico)	RESTORING (aplica logs)
4 · Último log	HotelDB_Log_YYYYMMDD_HHMMSS.trn (cola final)	RECOVERY	`RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_20250219_143000.trn' WITH RECOVERY;`	ONLINE (lista para producción)

Regla de oro de la recuperación ante desastres

- La cadena de recuperación (LSN) no debe romperse: aplicar el backup Full más reciente, luego únicamente el último Diferencial, seguido de todos los archivos de log generados con posterioridad al diferencial, en estricto orden cronológico.
- Todos los pasos previos se ejecutan con `WITH NORECOVERY`; solo el último comando de log incluye `WITH RECOVERY` para dejar la base de datos ONLINE.